

React.js Comprehensive Core Concepts Guide

A Complete Study & Practical Sheet for Frontend Developers

REFERENCE GUIDE & PRACTICAL SHEET

1. What is React and Why Do We Use It?

React (also known as React.js or ReactJS) is an open-source JavaScript library developed and maintained by Meta (Facebook) along with an active community of individual developers and companies. It is specifically designed for building modern, interactive, and high-performance user interfaces (UIs), particularly single-page applications (SPAs).

Key Reasons to Use React:

- **Component-Based Architecture:** UIs are broken down into self-contained, reusable pieces (components), making large applications maintainable and scalable.
- **Virtual DOM (Document Object Model):** React maintains an in-memory light-weight copy of the real DOM. When data changes, React efficiently updates and renders only the exact parts of the real DOM that modified, ensuring optimal performance.
- **Declarative Syntax:** React makes UI creation predictable and easier to debug by allowing developers to describe *what* the UI should look like for a given state.
- **Unidirectional Data Flow:** Data flows in one direction (from parent to child components via props), making state management predictable.
- **Rich Ecosystem & Community:** Huge ecosystem for state management (Redux, Zustand), routing (React Router), styling, and cross-platform mobile development (React Native).

2. React Components & Types of Components

Components are the fundamental building blocks of any React application. Every React visual element—a button, sidebar, form, or navbar—is rendered through components.

Feature / Aspect	Class Components (Legacy)	Functional Components (Modern Standard)
Definition	ES6 JavaScript classes extending <code>React.Component</code> .	Plain JavaScript functions that return JSX.
State Management	Uses <code>this.state</code> and <code>this.setState()</code> .	Uses the <code>useState()</code> Hook.
Lifecycle Methods	Uses <code>componentDidMount</code> , <code>componentDidUpdate</code> , etc.	Uses the <code>useEffect()</code> Hook.
Boilerplate & Syntax	More verbose, requires <code>this</code> binding.	Concise, modern, clean, and easy to read.

3. Functional Components: Practical Sheet

Functional components are standard JavaScript functions that accept `props` as an argument and return React Elements (JSX).

```
import React from 'react';

// Standard Function Declaration Component
function UserProfile() {
  return (
    <div className="profile-card">
      <h2>User Profile</h2>
      <p>Welcome back, React Developer!</p>
    </div>
  );
}

export default UserProfile;
```

4. Import and Export Statements

React heavily relies on ES6 module syntax (`import` and `export`) to organize code into separate files and promote reusability.

Types of Exports & How to Import:

A. Default Export

Used when a module exports a single main component or functionality. You can import it using any alias without curly braces.

```
// --- Button.jsx (Exporting) ---
const Button = () => <button>Click Me</button>;
export default Button;

// --- App.jsx (Importing) ---
import CustomButton from './Button'; // Custom name allowed
```

B. Named Export

Used when a module exports multiple utilities, components, or constants. Imports **must** match the exported name inside curly braces `{ }`.

```
// --- Helper.jsx (Exporting multiple items) ---
export const Header = () => <header>Header Content</header>;
export const Footer = () => <footer>Footer Content</footer>;

// --- App.jsx (Importing) ---
import { Header, Footer } from './Helper';
```

5. Component Props (Parent & Child Components)

Props (short for properties) are read-only inputs passed from a Parent Component to a Child Component to render dynamic content.

```
// --- Child Component (Greeting.jsx) ---
function Greeting(props) {
  return <h1>Hello, {props.name}! You are {props.age} years old.</h1>;
}

// Alternative using Destructuring (Best Practice):
function GreetingDeconstructed({ name, age }) {
  return <h1>Hello, {name}! You are {age} years old.</h1>;
}

// --- Parent Component (App.jsx) ---
function App() {
  return (
    <div>
      <Greeting name="Alice" age={25} />
      <Greeting name="Bob" age={30} />
    </div>
  );
}
```

6. Function Expressions, Arrow Functions & Anonymous Functions

React functional components can be written in several JavaScript syntax variations:

1. Function Declaration

```
function Card() { return <div>Card</div>; }
```

2. Function Expression

```
const Card = function() { return <div>Card</div>; };
```

3. Arrow Function (Industry Standard)

```
const Card = () => {
  return <div>Card</div>;
};

// Implicit Return Short Syntax:
const MiniCard = () => <div>Mini Card</div>;
```

4. Inline / Anonymous Function inside JSX

Frequently used in event handling:

```
<button onClick={() => alert('Button Clicked!')}>Click</button>
```

7. Embedded JSX Expressions

JSX allows embedding any valid JavaScript expression (variables, mathematical operations, function calls) directly inside markup using **curly braces** `{ }`.

```
const UserCard = () => {
  const username = "Jane Doe";
  const userRole = "Admin";
  const calculateScore = (base) => base * 1.5;

  return (
    <div>
      <h3>User: {username.toUpperCase()}</h3>
      <p>Role: {userRole}</p>
      <p>Calculated Score: {calculateScore(100)}</p>
    </div>
  );
};
```

8. Conditional Rendering with Ternary Operator

The **ternary operator** (`condition ? true : false`) is the standard method for conditional rendering inside embedded JSX expressions.

```
const UserStatus = ({ isLoggedIn, username }) => {
  return (
    <div>
      {isLoggedIn ? (
        <h2>Welcome back, {username}!</h2>
      ) : (
        <h2>Please log in to continue.</h2>
      )}
    </div>
  );
};
```

9. Different Ways to Style JSX

React offers several flexible approaches for styling components depending on application scale and architecture.

1. Inline Styles

Written as JavaScript objects using camelCase CSS property names.

```
const headerStyle = {
  backgroundColor: '#0f172a',
  color: '#38bdf8',
  padding: '10px'
};

const Header = () => <h1 style={headerStyle}>Inline Styled Header</h1>;
```

2. External CSS Stylesheets

Standard `.css` files imported directly into components.

```
// styles.css -> .btn { background-color: blue; color: white; }
import './styles.css';

const Button = () => <button className="btn">Click</button>;
```

3. CSS Modules

Scoped styling solution preventing CSS class name collisions (e.g., `Button.module.css`).

```
import styles from './Button.module.css';

const Button = () => <button className={styles.primaryBtn}>Scoped Button</button>;
```

4. CSS-in-JS Libraries & Utility Frameworks

- **Styled Components / Emotion:** Write actual CSS directly inside JS files using tagged template literals.
- **Tailwind CSS:** Utility-first framework applying class names directly (e.g., `className="bg-slate-900 text-sky-400 p-4"`).

Summary Checklist & Best Practices

- Prefer **Functional Components** with modern ES6 arrow functions for clean code.
- Use **Default Exports** for component files and **Named Exports** for utility libraries/constants.
- Use **Destructuring** for component props to make parameters clean and explicit.
- Leverage **JSX Expressions** `{}` and the **Ternary Operator** for clean inline logic.
- Select styling methods based on team needs: CSS Modules/Tailwind for scalable production apps.